



ROBOCUPJUNIOR RESCUE LINE 2026

TEAM DESCRIPTION PAPER

QUANTUMSPEED

Matteo Defant, Luca Facchinelli, Simone Bettega

Abstract

The robot is designed to compete in the RoboCupJunior Rescue Line. We have been working for almost three years to create a robot capable of overcoming all the challenges presented by the competition. Over time, the robot has undergone numerous modifications: we have added artificial intelligence systems; we have increased the number of sensors; we have used better and more suitable sensors for our goals and we have furthermore improved our mechanical components by the addition of a brand new tire system built from scratch. Thanks to these improvements, the robot has significantly enhanced its performance, demonstrating greater reliability in handling complex line-following sections and improved capability in overcoming physically challenging elements of the course. The entire robot was designed in 3D using *Fusion 360*.

To achieve greater reliability, we decided to relay only on one microcontroller, getting rid of the old serial communication system. In fact, all the computations are done by a Raspberry Pi 5, programmed in Python, connected to a Pi camera, which is used to identify and follow the line, using the tools provided by the *OpenCV* library, additionally the camera is used to manage the evacuation zone, recognising the entrance and the exit of it. The raspberry is also connected to an AI hat, using a model trained with TensorFlow Lite, it is able to recognize victims and evacuation points in the evacuation zone using a second vertical Pi camera.

1. Introduction

a. Team

- **Matteo Defant**, the team captain, is currently responsible for the mechanical design of the robot using Fusion 360 and overseeing the physical assembly. Furthermore, he is in charge of programming the evacuation zone logic. Building on his previous experience, he continues to coordinate the project's technical development. He has previously participated in the RoboCupJunior Rescue Line in 2024 and 2025 qualifying for the european championship competition.
- **Luca Facchinelli**, manages the line-following system, utilizing vision-based technology and a PID algorithm to ensure precise navigation. He continues to refine the robot's movement logic to optimize performance on the track. He has previously participated in the RoboCupJunior Rescue Line in 2024 and 2025 qualifying for the european championship competition.
- **Simone Bettega**, oversees the system integration and the detection of the silver strip. He works closely with Luca on the development and tuning of the line-following code. He is specifically responsible for the development of the "lack of progress" procedure, implementing robust file services and utilizing multiprocessing. He has previously participated in the RoboCupJunior Rescue Line in 2024 and 2025 qualifying for the european championship competition.

2. Project Planning

a. Overall Project Plan

Team's objective

Our goal, since we've now been working on the robot for several years, is to achieve a strong result at the World Cup, due to the fact that this is our last possibility to achieve an international title greater than the previous second place at the european championship. To reach this goal, the robot must be able to overcome all the challenges faced in the previous years, including the limitations imposed by the previous line detection method (relying on a QTR sensor). So far, we have been able to overcome most of them, but not all. Nevertheless, we have focused on the robot's speed and reliability in order to still achieve a good score. Another major challenge has been the design and development of the robot's physical components.

These are the various difficulties and their respective solutions:

- **Intersections and Green markers:** Intersections analysed at different angles with the camera can be tricky to detect: the main mask struggles to recognise a clean contour, on top of that the green markers are not always easy to orient with respect to the center of the intersection. The solution stands in validation methods exploiting the intersections between the main lines and the borders of the frame recorded.

- **line detecton:** the robot needs to identify a unitary area comprehending the balck line, which is not always easy when it comes to different enviroments (especially when it comes to illuminations conditions). In order to solve that we used various tools provided by OpenCV, including matrices operations (kernel) and adjustmens in the camera settings (such as contrast and saturation).
- **Silver line:** The main obstacle with the recognition of a reflective object was the sudden changes of the values read in an hsv scale, due to the change of illumination, an approach using image processing was no longer reliable; for that reason we switched to an object detection recognition based on *yolo26n* and trained using the ultralytics library
- **Obstacle:** the robot detects the obstacle using a TOF sensor placed in front of it. As soon as it detects the obstacle, it navigates around it until it finds the line again constantly adjusting itself using the sensors on the side of the robot.
- **Bump:** The new silicone tire system has proved to be very efficient in overcoming this issue, especially when it comes to bump being placed in high slope regions which was a main obstacle in the beginning.
- **Ascents, descents and inclined paths:** initially, this was a major challenge as well, in spite of that, by the use of a gyroscope the robot detects the absolute angle in 9 axis, and with that we are able to change the correction sent to the motors accordingly, the silicone tires also helped to improve stability at greater slopes.
- **Victim and evacuation point recognition:** to recognize the spheres, we use a Raspberry Pi 5 trained with TensorFlow Lite. Combined with two TOF sensors, we detect the presence of obstacles or objects between the robot and the target object.
- **Sphere collection:** to collect the spheres, we use a mechanical arm that picks up and sorts each sphere into a chest and than it proceeds to delivery them all at a time, significantly reducing the time spent in the evacuation zone.
- **Exit recognition:** we tried to recognize it separatly, but in the end, we decided to follow the edge using TOF sensors and a gyroscope and to detect the lack of a silver strip instead.

Overall project plan

All the solutions to the problems listed above have been tried and tested multiple times. Additionally, there were several ideas and changes throughout the year, leading us to better solutions. Below is our project plan, divided by months, with the objectives we set at the beginning of the year. At the end of each month, we wrote a comment and the objectives we actually achieved.

- **September:** Met to combine ideas; understood what needed improvement and created a prototype of the robot. The individual tasks were:
 - Matteo: Design the robot in 3D and determine if it was possible to collect all the spheres at the same time.
 - Simone: Create a line-following PID algorithm using the camera.
 - Luca: Create a line-following PID algorithm using the camera.

“Fortunately, we managed to achieve all our goals by the end of September; a first prototype of the robot has been printed, so the line-following system can be tested on the field from now on.”

- **October:** Continued making improvements to reach the final version of the robot, so we could test the line-following algorithm with stable lighting and start programming the evacuation zone.
 - Matteo: Assemble the robot with the sensors and all the printed components, and begin programming the evacuation zone.
 - Simone: Develop a ”lack of progress” procedure.
 - Luca: Improve the PID algorithm and introduce green detection.

“The robot and the line-following system work, but the robot shows some mechanical problems in overcoming bumps on ramps. However, it’s not a major problem yet, so we will keep the November objectives unchanged.”



- **November:** Finalized a first version of the complete code, merging the PID algorithm with the evacuation zone code.
 - Matteo: Merge the codes and develop a first version of the evacuation zone code.
 - Luca: Create the final line-following system and develop an obstacle-avoiding algorithm.
 - Simone: Create a silver strip detection algorithm.

“The silver detection algorithm using HSV with OpenCV is unreliable and has false detections with bumps, so the codes have not been merged.”

- **December:** Aimed for a working robot capable of overcoming most challenges.
 - Matteo and Luca: Merge their programs and resolve the various issues.
 - Simone: Find a way to improve the silver strip detection.

“The codes are now merged, the silver strip detection is still not completely reliable, and complete tests are being carried out.”

- **January and February:** Started programming the evacuation zone exit and improved the evacuation zone logic.
 - Matteo: Work on the evacuation zone logic.
 - Luca: Program the evacuation zone exit.
 - Simone: Test and complete the “lack of progress” procedure.

“The “lack of progress” algorithm is now complete and fully functioning; a first version of the exit is ready.”

- **March:** Made a difficult decision to win the regional competitions.
 - All: Unpack the old robot and fine-tune it for the regional competition.

“Even though we achieved significant results with the new robot, we accepted the fact that the old robot was more performing at that moment, so we decided to use it for the regional competition; in the end, we won the regional selection.”

- **April:** Solved the issues from the regionals and qualified for the World Championship.
 - Luca: Improve and, if necessary, change the techniques to overcome the difficulties in line-following.
 - Matteo: Fine-tune the evacuation zone logic.
 - Simone: Solve the mechanical problems that previously emerged.

“During the first days of the national competition, we had several problems, but after a couple of sleepless nights, we managed to find a temporary solution for the problems and ended the national competition in second place, qualifying for the World Championship in Incheon.”

- **May:** Prepared to achieve a good score in the World Championship and assessed the level of other teams and robots.
 - Matteo: Rebuild the robot, change the sensors, and improve the mechanics.
 - Luca: Solve the problems the robot encountered in the national competitions.
 - Simone: Train a new AI model to detect the silver strip.

“During the national competition, we encountered problems related to the evacuation exit, bumps on ramps, and silver detection, forcing us to develop a new algorithm.”

- **June:** Prepared to achieve a good score in the World Championship and finalized the documentation.
 - Matteo: Prepare the technical video and introduce the AI accelerator hat.
 - Luca: Prepare the poster and solve minor problems related to the PID.
 - Simone: Prepare the TDP and conclude the AI detection for the silver strip.

“Due to final exams, we did not have much time and the progress rate slowed down, however, most of the problems are being overcome.”

b. Integration Plan

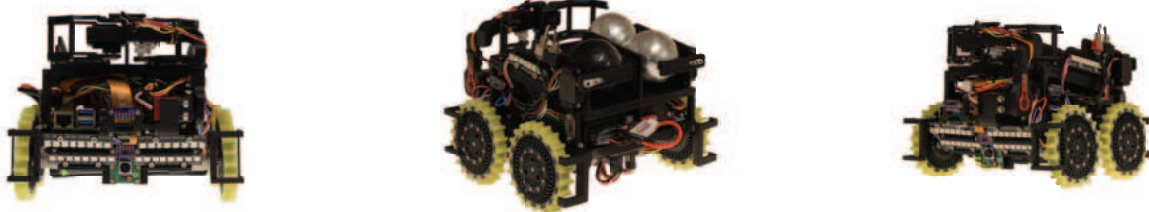
Our robot relies on a Raspberry Pi 5, equipped with an AI Hat, to handle complex tasks like computer vision and decision-making. To manage hardware interaction, the Raspberry Pi communicates with a servo driver for the five actuators used in victim pick-up and release, and utilizes a multiplexer to interface with the five Time-of-Flight (ToF) sensors and the gyroscope via the I2C bus. This setup ensures efficient data flow for navigation and environmental awareness. For propulsion, we employ a dedicated motor driver powered by a separate 11.1V battery boosted to 12V. This ensures the motors have consistent torque, independent of the logic and sensor circuits, which are powered by the 7.4V battery through dedicated DC-DC converters to maintain stable voltage levels. Before finalizing each component, we prioritized reliability and performance. For example, we integrated high-speed ToF sensors over ultrasonic ones to minimize latency, and we optimized our sensor selection to ensure response times is under 250ms. This rigorous approach, from electrical design to signal processing, allows the robot to handle both high-level AI tasks and low-level motor control with the necessary speed and precision.

3. Hardware

As already mentioned, the robot has been entirely designed and printed in PLA by us, which allows us to adapt it to our needs and makes it easier to modify the various components. After various designs and prototypes, we reached the final version. It is printed as compact as possible to prevent the robot from getting stuck during the course. The base, or chassis, of the robot is a 15 x 16 cm rectangle with a thickness of one centimeter. The four motors, the main battery and camera for object detection are mounted underneath the base. On the upper side, we have all the other electronic components. Additionally, on the top, we can find the handle and the rescue mechanism used to collect the spheres. There are also hollow spaces inside the robot to keep the jumpers organized, reducing the likelihood of them becoming detached. The overall dimensions of the robot are approximately 20cm x 20cm x 20cm.

a. Mechanical Design and Manufacturing

Main structure



The robot's structure has been studied and modified multiple times. The first prototype had 3 wheels, but we later realized that a robot with four wheels performed significantly better. We now use four driven wheels, and it was necessary to test various types of wheels. In the end, we decided to use wheels covered with a layer of silicone, which ensures excellent traction on all surfaces. The friction is so high that many times the robot would "climb" the walls of the evacuation zone. To address this, we decided to install bumpers around the front wheels. Furthermore, we had to round the bumpers, as they risked getting stuck and blocking the robot. Additionally, it was crucial to design the bottom part of the robot well because the supports for the sensors, batteries and motors, tended to bend. Despite all these challenges, we managed to achieve a structurally effective robot.

Rescue mechanism



The mechanism for collecting the victims was also entirely designed by us. It consists of an arm, powered by three servo motor in total: two independent motors are moving the gripping mechanism and the other one is rotating the whole assembly in order to store the spheres into their specific slot. This type of design improves the previous one which was unable to sort each sphere into a specific slot. Furthermore, the gripping mechanism has been significantly improved through the introduction of two microswitches. These are used to provide tactile feedback and detect the correct pick-up of the sphere, ensuring that the robot successfully secures the victim before proceeding to the storing.

b. Electronic design and manufacturing

The robot is powered by two OVONIC LiPo batteries. The first battery (5200mAh, 7.4V) powers the Raspberry Pi 5 and the AI Hat, while the second one (2200mAh, 11.1V) is dedicated to the motion system. Since the Raspberry Pi requires a stable 5.1V supply, a DC/DC converter is used to step down the 7.4V input, preventing potential damage. For the 11.1V battery, we use a dedicated DC/DC converter to provide 5V for the servo driver, and a step-up booster to regulate the voltage to 12V for the motor driver. This separation prevents voltage drops that previously caused system casual reboots.

We chose a dual-battery setup because a single battery couldn't handle the high current demands of the entire robot. By separating the power sources, we ensure that the high-load components, like the motors and servos, have a dedicated supply that doesn't interfere with the Raspberry Pi's power, preventing voltage drops and unexpected reboots during operation.

To build the robot, we used the following electronic components:

- Raspberry Pi 5 (8GB RAM) + AI Hat: handles computer vision and high-level logic.
- Servo Driver: manages the 5 servos for victim pick-up and release.
- Motor Driver: controls the DC motors with 12V power.
- Multiplexer: allows the Raspberry Pi to interface with 5 ToF sensors and the gyroscope via I2C.
- 4 DC motors (powered at 12V).
- 5 ToF VL53L0X sensors: for obstacle detection and navigation.
- 1 MPU6050 gyroscope: for precise heading control.
- 5 Servomotors: for the mechanical arm and victim handling.
- 2 DC-DC step-down converters.
- 1 Step-up booster (11.1V to 12V).
- 2 Raspberry Pi Camera Modules 3: for vision-based line following and victim detection.
- 6 Microswitches: for detecting the correct pick-up of spheres.

- 1 5V LED strip.
- 1 OVONIC 5200mAh 7.4V battery.
- 1 OVONIC 2200mAh 11.1V battery.
- 1 General power switch.

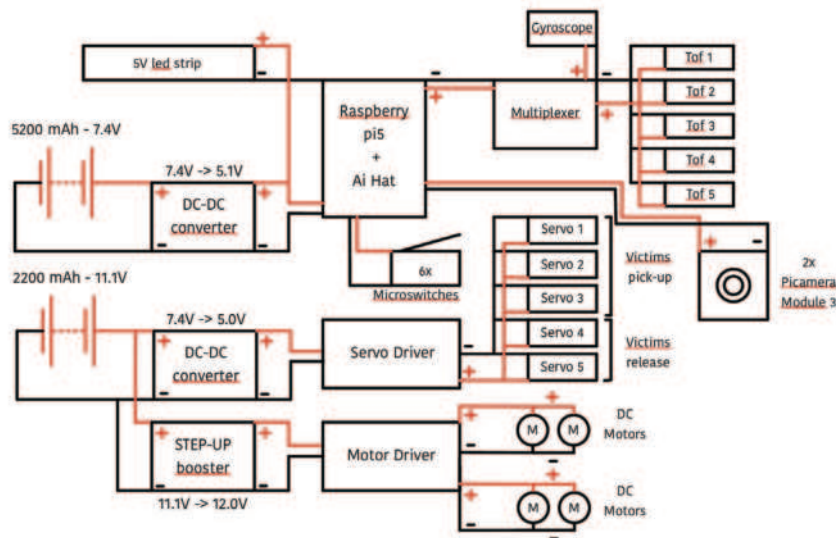


Figure 3: Qualitative electrical diagram

c. Innovative solutions

Obstacle Detection By increasing the reliability in detecting it, the use of switches allows for a double check to confirm the presence of any object placed on the path. The mechanical component serves both as a physical constraint, which positions the robot in a frontal position relative to its target, and as a means to ensure accurate distance measurement, for which we found a lack of precision when using the TOF sensor.

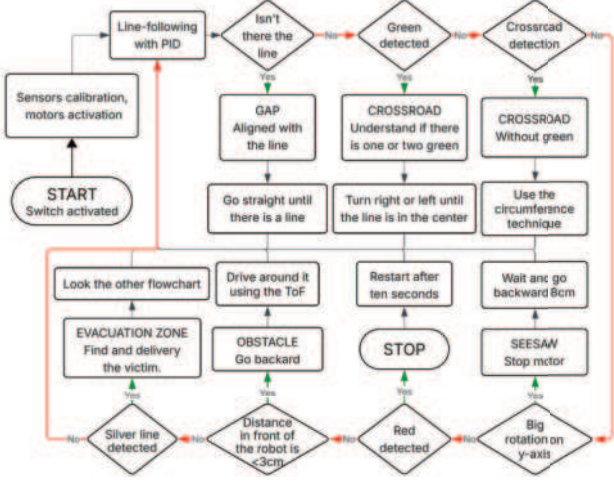
Simultaneous collection mechanism In the front part of the robot, we have integrated a custom-designed collection mechanism capable of gathering all spheres simultaneously. This system allows the robot to efficiently clear the evacuation zone without needing multiple passes, significantly reducing the time spent in the area.

4. Software

Software Architecture: As previously explained, the entire project is programmed in Python using Visual Studio Code, leveraging the Raspberry Pi 5's processing power. Instead of relying on a secondary microcontroller, the code is structured around a multiprocessing architecture. This allows the robot to handle critical tasks simultaneously: the main process manages the computer vision for object and color recognition, while parallel processes handle the PID logic for line-following, sensor data acquisition via the multiplexer, and the control of actuators for victim handling. All processed data is integrated in real-time to ensure precise navigation and decision-making. In the evacuation zone, the system synchronizes these processes to efficiently collect spheres and navigate to the drop-off points. Below is a flowchart explaining how the program logic is structured.

a. General software architecture

Given the complexity of the code, we have included a simplified flowchart to understand when the robot performs all the functions. For this reason, we will divide this section into two:



(a) Line follower chart



(b) Evacuation zone chart

Line follower

- The PID (Proportional–Integral–Derivative) algorithm, a central and most widely used component throughout the competition (except for the room), is used to calculate a correction to be applied to the robot's motors in order to keep the robot centered on the line. With the Picamera and OpenCV, a library of Python, we can calculate the distance between the center of the robot and the line, and the goal of the algorithm is to minimize the error between this position and the ideal position (typically the center of the frame).

The error is calculated as:

$$e(t) = \text{ideal_location} - \text{current_location}$$

The PID correction is then calculated using the following formula:

$$u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau) d\tau + K_d \cdot \frac{de(t)}{dt}$$

Where:

- $u(t)$ is the correction to be applied; that is, the speed variation between the left and right motors.
- $e(t)$ is the instantaneous error.
- t^-, i^- are the previous values.
- K_p, K_i, K_d are, respectively, the proportional, integral, and derivative coefficients.

In the code, we used a simplified formula, which is as follows:

$$u[t] = K_p \cdot e(t) + K_i \cdot \sum_{i=0}^t (e(i) - e(i^-)) + K_d \cdot (e(t) - e(t^-))$$

Subsequently, this correction is multiplied by a fourth and final factor, introduced to integrate the analysis of the second dimension (indeed, previous calculations exclusively considered the difference between the X coordinates

of the points). This factor is based on the difference of the Y coordinates raised to the power of 2.11 and multiplied by a constant. To this term, 1 is added: this way, if the difference in Y is zero, the factor takes a value of 1, leaving the correction unchanged rather than canceling it out. The final formula is:

$$u[t] = [K_p \cdot e_x(t) + K_i \cdot \sum_{i=0}^t (e_x(i) - e_x(i^-)) + K_d \cdot (e_x(t) - e_x(t^-))] \cdot [e_y(t)^{2.11} \cdot K_y + 1]$$

- **Color Recognition:** Using the OpenCV library, we can create a mask for each color. Before validating the presence of the target color, we verify that its area exceeds a predefined threshold. To define a mask, we need to initialize two NumPy arrays containing the minimum and maximum HSV values for that specific color; for example, these are the thresholds we use for green:

$$\text{greenL} = \text{np.array}([40, 90, 70]) \quad \text{e} \quad \text{greenU} = \text{np.array}([80, 255, 255])$$

This method reduces the possible errors caused by lighting or other factors.

- **Tilt recognition:** This is done using an lsm6dsox-lis3mdl gyroscope; a sensor that returns the tilt with respect to all three axes. By interpreting this value, it is possible to understand the tilt of the robot relative to the ground. We use it to detect if the robot is going uphill, to know how many degrees it has rotated during a turn, and to straighten the robot if it is at risk of tipping over. The gyroscope is also used to detect when the robot has just navigated a ramp (or seesaw), allowing it to stop and realign with the line.
- **Room orientation:** Using five TOF sensors, the robot can determine its distance from the walls in the evacuation zone. It can also identify the evacuation points and avoid them when it is searching for the exit from the room. Furthermore, it can also detect obstacles and bypass them.
- **Motor management:** The motors are driven in pairs, and their voltage (and thus their speed) is adjusted based on the correction provided by the PID. In the case of predefined algorithms, such as the obstacle turn, the motors are driven with values found experimentally. To facilitate the writing of these algorithms, basic functions (right, left, forward, and backward) have been implemented in the code.

Evacuation zone

- **Recognition of “victims” and release zones:** The robot identifies “victims”—defined as black and silver spheres—using a machine learning model trained on a dataset of over two thousand images. During navigation, the camera captures video frames in real-time, allowing the model to detect and localize the spheres. Based on the object’s spatial coordinates within the frame, the control system adjusts the motors to intercept and collect the spheres. To ensure high-level performance, the model inference is accelerated using a Raspberry Pi AI HAT+ (13 TOPS).
- **Release zone recognition** is implemented via HSV color space masking using the OpenCV library.
- **Obstacle and evacuation point distinction:** The robot distinguishes between obstacles and evacuation points using a two factor validation system. Limit switches (microswitches) mounted on the chassis are triggered upon physical contact to identify obstacles. The robot uses its onboard camera to calculate the distance to the evacuation point, enabling it to differentiate between the two and determine whether to drop the sphere or navigate around the obstacle.

b. Innovative solutions

Line follower

One of the most innovative solutions was the room recognition through a long gap. The robot understands that it has entered the room when the gap is too long and thus out of bounds, so it must be the entrance to the room. This solution is much better than many other approaches, as it is not guaranteed that the silver line will lead to the correct spot.

Additionally, another solution is that instead of constantly checking for the presence of an obstacle, we only check

it approximately every 0.5 seconds. This way, we avoid slowing down the rest of the code.

Another solution is obstacle recognition using TOF sensors and the use of two side buttons, because it's not guaranteed that the obstacle will be in the center of the line; it could also be on the side, and in that case, the TOF sensor wouldn't detect it.

Evacuation zone

The most innovative part of the code is undoubtedly the image recognition algorithm, developed to meet the need for reliable recognition regardless of the environment in which the robot is located. Factors such as lighting and variability in the appearance of competition objects do not significantly affect the algorithm we used.

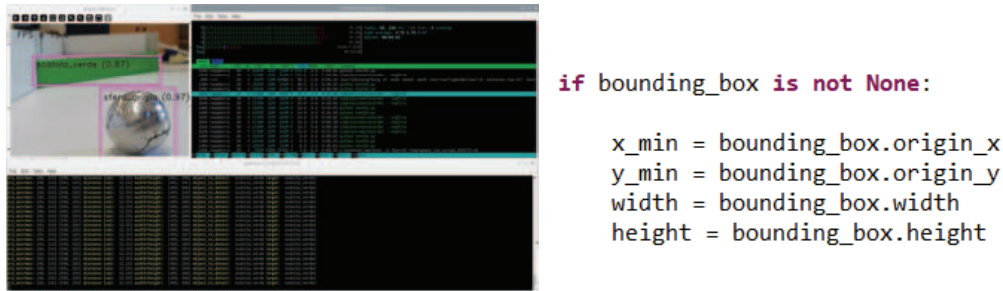


Figure 5: Example of the code and the camera view

General information about TensorFlow, computation speed, and advantages.

The “Lite” variant of TensorFlow allows for high performance on devices with limited memory or hardware constraints in general. We selected it among all the competing models as the most suitable for running on a Raspberry Pi 5.

Ultimately, it is essential to use a model with a high number of FPS to ensure the most responsive feedback possible. Inside the room, the system receives a large amount of data, which is then used to calculate the error (interpreted through the PID algorithm) that allows the robot to orient itself toward the object recognized by the model.

Model Operation and Bounding Boxes

As mentioned earlier, the model operates with a high frequency of images. Specifically, the Pi-camera captures frames that are then broken down into matrices that can be analyzed by the model, which always returns an affinity value as output. The ultimate goal of this analysis is to orient the robot towards the target as quickly as possible.

Model training

One of the most significant challenges leading to the finalization of the methodology used for sphere recognition was certainly the “training” phase. Among the various methods available to train the model, we identified supervised training as the most accessible variant given our initial skills.

This method requires image training, where direct user (trainer) intervention is needed. The user is responsible for taking images in suitable environments, ensuring variations in lighting, exposure, and other elements that will influence the model's flexibility.

The “bounding boxes” are manually placed at points of interest for each image. Although this method is somewhat repetitive, it provides a more user-friendly approach, making it more effective in understanding the model.

The “bounding box” is essentially a rectangle: The calculation of the four coordinates is aimed at orienting the object in relation to our reference, which is the rest of the matrix. This operation is performed simultaneously for each object that the model can recognize. The final result is a matrix containing the coordinates for all the objects recognized within the frame.

Setting up the model

Once inside the room, the robot begins the preparation phase aimed at sequentially identifying all the targets. First of all, global variables are declared within the code, which are fundamental parameters for the proper functioning:

One of the most impactful is certainly the “value threshold” variable. It determines the minimum acceptable affinity percentage relative to the one returned by the model. This allowed us to have a “smoother” model, capable of adapting to every environment and situation in which it operates.

Conversion to “real” distance

Once the object is identified, it is necessary, in addition to its dimensions, to estimate its distance. This allows us to coordinate the motors to bring the robot as close as possible to the victim. Once the target distance is reached, the robot proceeds with a script, where the mechanical arm will collect the sphere and then move on to the delivery protocol. To calculate the real distance of the object, it is necessary to use a conversion factor. By referring to the size of the bounding box, specifically its width, we were able to empirically derive two variables: $focalLength = 1166$ and $realWidth = 4.828$; these were then applied in the formula that describes the real distance of the sphere:

$$Distance = \frac{focalLength \cdot realWidth}{width}$$

Where:

- $focalLength$ is the distance that separates the optical center from the sensor plane.
- $realWidth$ is the actual width of the real object.
- $width$ is the width of the object, calculated using the x coordinates of the bounding box.

The robot enters the collection phase only when: $Distance < 20\text{cm}$, thus ensuring a proper grip by the mechanical arm, which will then perform an additional check via a button placed inside the “cage”.

5. Performance evaluation

As mentioned previously, the robot was completely built by us, from the design phase to the 3D printing. This allowed us to have great control over every physical component of the robot. Every component, whether made or purchased, must undergo a series of tests and checks before being permanently assembled. Testing the reliability and performance of the various robot parts is one of the most important aspects of the work. After constructing and assembling the components in the most effective positions, it's necessary to test and observe the values read by the sensors. It's important that no sensor is influenced by another, which is why it is tested in various scenarios. Dozens of hours have been spent testing and improving all variables in the software, starting with the correct constants for the PID controller and reaching the right sensitivity for detecting victims in the room. The only way to achieve the ideal values is by continuously testing. Furthermore, it's important to change the logic of the code to understand what is the best approach and to discover all the robot's capabilities. Additionally, given the use of sensors, it's even more important to use the correct parameters, as even one wrong setting could ruin the robot's performance. This whole process helped us bring the robot to a high level.

6. Conclusion

All the team members have already had this experience, but this year has been the best because we were able to work well as a team. We learned a lot, both about programming and teamwork. We solved many problems with the robot, and above all, we managed to build a robot that is much more competitive compared to previous years, developing systems that we wouldn't have even imagined in the past. Satisfied with the robot we built, we are excited and proud to go to the World Championship competition, both to race on the most challenging fields and to compare ourselves with other teams from all over the world.

7. References

- Raspberry “www.raspberrypi.com”
- Bambu Lab “eu.store.bambulab.com”
- Pololu “www.pololu.com”
- Ovonix “www.ampow.com”
- Adafruit “www.adafruit.com”
- Python “www.python.org”
- Tensorflow Lite “<https://github.com/tensorflow/tensorflow>”
- Yolo26n “<https://github.com/tensorflow/tensorflow>”

Components:

- Motors DC 12V “<https://www.tme.eu/it/details/pololu-3480/motori-dc/pololu/156-1-metal-gearmotor-20dx44l-mm-12v-cb/>”
- TOF sensors “www.az-delivery.de/it/products/vl53l0x-time-of-flight-tof-laser-abstandssensor”
- Motorschild “<https://amzn.eu/d/0aP4tPBq>”
- I2C Multiplexer “<https://amzn.eu/d/0csN2iZ3>”
- MPU6050 “www.robotstore.it/Modulo-GY-521-MPU-6050”
- Servo motor “www.informaticait.it/elettronica/servomotori/Servo-motore-servomotor-MG995-tower-Pro-55g-coppia-13kg-ingranaggi-in-metallo.html”
- DC-DC step-down “www.amazon.it/Converter-Binghe-Convertitore-Alimentatore-1-25-35V/dp/B0D7BWSTCX”
- Raspberry Pi camera v3 wide “<https://meloper.com/it/shop/accessories/cameras/camera-module-3-wide/>”
- Step Up module “<https://amzn.eu/d/00vWHN63>”

Special thanks to:

